```
=>
#<Product:0x0000004013a47ad8
...
irb(main):002:0> p.title
=> "Modern CSS with Tailwind"
irb(main):003:0> p.price = 29.00
=> 29.0
irb(main):003:0> p.save
=> true
```

Once we have a console session open, we can poke and prod all the various methods on our models. We can create, inspect, and delete records. In a way, it's like having a root console to your application.

Once we put an application into production, we need to take care of a few chores to keep the application running smoothly. These chores aren't automatically taken care of for us, but luckily we can automate them.

## Dealing with Log Files

As an application runs, it constantly adds data to its log file. Eventually, the log files can grow extremely large. To overcome this, most logging solutions can *roll over* log files to create a progressive set of log files of increasing age. This breaks up our log files into manageable chunks that can be archived or even deleted after a certain amount of time has passed.

The `Logger` class supports rollover. We need to specify how many (or how often) log files we want and the size of each, using a line like one of the following in the file `config/environments/production.rb`:

```
config.logger = Logger.new(config.paths['log'].first, 'daily')
```

Or this is a possibility:

```
require 'active_support/core_ext/numeric/bytes'
config.logger = Logger.new(config.paths['log'].first, 10, 10.megabytes)
```

Note that in this case an explicit require of `active_support` is needed, because this statement is processed early in the initialization of your application—before the Active Support libraries have been included. In fact, one of the configuration